Homework 5: Ontology-based Semantic Grounding

Group 10 Semantic Affordance Grounding

Group 10
蔡尚融

AI Capstone 2026

Abstract

This report describes the Group 10 submission for Homework 5. The project builds a compact ontology for grounding baseline task objects in the AI Capstone environment and inferring which objects are graspable by a robot gripper. The workflow uses RDF/OWL for modeling, OWL/RDFS closure plus a documented rule layer for reasoning, and SPARQL for querying inferred graspable objects. The implementation is reproducible with `uv`, RDFLib, and `owlrl`.

1 Project Overview

The goal of this homework is to create a semantic grounding layer for a robot manipulation environment. Instead of representing objects only as visual labels, the ontology represents each object as a typed individual with a task role, affordances, object labels, colors, and pose-frame identifiers. The reasoning workflow then infers `cap:GraspableObject` membership from the presence of a grasping affordance.

The repository contains the required ontology files, inferred graph, SPARQL queries, saved query outputs, source code, tests, README, and this report. The baseline tasks modeled in the ontology are:

- Cup stacking: blue cup and pink cup.
- Cutlery arrangement: knife, fork, and plate.
- Toy block collection: toy blocks and basket.

2 Repository Contents

3 Namespace Policy

The shared course namespace is:

```
@prefix cap: <https://hcis.io/ontology/aicapstone/2026/> .
```

The Group 10 namespace is:

```
@prefix g10: <https://hcis.io/ontology/aicapstone/2026/group10/> .
```

Course-level classes and properties are reused from `cap:`. Group-specific individuals, tasks, affordance individuals, and role individuals are declared under `g10:`. For example, `g10:blueCup01` is a group-specific object instance, while `cap:Cup` is a shared course class.

Task roles are represented as Group 10 role individuals. For example, `g10:targetObjectRole` is typed as `cap:TargetObject`, and object instances point to that individual through `cap:hasTaskRole`. This avoids using OWL classes directly as property values.

Path	Purpose
ontology/group-ontology.ttl	Group 10 authored ontology with task objects, task roles, affordances, and the formal graspability axiom.
ontology/imports/course-affordance.ttl	Provided course ontology, with a minimal Turtle syntax repair for cap:hasApproxWidth.
ontology/imports/course-alignment.ttl	Provided SKOS alignment resource for semantic traceability.
ontology/inferred-results.ttl	Exported graph after reasoning.
queries/graspable_objects.rq	Required SPARQL query for inferred graspable objects.
queries/task_objects.rq	Additional query for all modeled baseline task objects.
results/graspable_objects_output.txt	Saved output of the required query.
src/run_reasoning.py	Reproducible local reasoning and query workflow.
tests/test_hw5_workflow.py	Tests for parsing, modeling constraints, inference, and saved outputs.

Table 1: Main repository files for the HW5 submission.

4 Ontology Design

The ontology separates object type, task role, affordance, and instance. This distinction is important because task relevance is not identical to graspability. For example, the plate is relevant as a reference object in cutlery arrangement, and the basket is relevant as a container target in toy block collection, but neither is modeled as a direct grasp target in this baseline submission.

Layer	Meaning	Examples
Object type	Shared class for object category	cap:Cup, cap:Knife, cap:ToyBlock
Task role	Role played in a task	cap:TargetObject, cap:ReferenceObject
Affordance	Action possibility in context	cap:GraspingAffordance, cap:ContainmentAffordance
Instance	Group-specific observed object	g10:blueCup01, g10:block01

Table 2: Ontology modeling layers.

Instance	Type	Role	Affordance
g10:blueCup01	cap:Cup	cap:TargetObject	grasping, stackability
g10:pinkCup01	cap:Cup	cap:TargetObject	grasping, stackability
g10:knife01	cap:Knife	cap:TargetObject	grasping
g10:fork01	cap:Fork	cap:TargetObject	grasping
g10:plate01	cap:Plate	cap:ReferenceObject	support
g10:block01	cap:ToyBlock	cap:CollectableObject	grasping
g10:block02	cap:ToyBlock	cap:CollectableObject	grasping
g10:basket01	cap:Basket	cap:ContainerTarget	containment

Table 3: Baseline task objects modeled by Group 10.

5 Reasoning Pattern

The central inferred class is cap:GraspableObject. The provided course resources already refer to this course-level term, and Group 10 adds its formal class definition because the starter affordance ontology does not include the full OWL axiom. The definition follows the homework specification: a graspable object is a physical object that has at least one grasping affordance.

```
cap:GraspableObject
    a owl:Class ;
    rdfs:subClassOf cap:PhysicalObject ;
    owl:equivalentClass [
        a owl:Class ;
        owl:intersectionOf (
            cap:PhysicalObject
            [
                a owl:Restriction ;
                owl:onProperty cap:hasAffordance ;
                owl:someValuesFrom cap:GraspingAffordance
            ]
        )
    ] .
```

The source group ontology does not manually assert Group 10 objects as `cap:GraspableObject`. The local workflow performs the following steps:

1. Load the course affordance ontology.
2. Load the course SKOS alignment ontology for traceability.
3. Load the Group 10 ontology.
4. Apply OWL/RDFS closure with `owlrl`.
5. Materialize the assignment-specific graspability rule for physical objects with a grasping affordance.
6. Export `ontology/inferred-results.ttl`.
7. Run SPARQL queries and save outputs under `results/`.

The additional materialization rule is documented because RDFLib and `owlrl` provide a lightweight reasoning workflow rather than a complete OWL DL reasoner. This keeps the inference reproducible while still matching the required homework pattern.

6 SPARQL Query and Results

The required query retrieves inferred `cap:GraspableObject` individuals in the Group 10 namespace and reports each object's object label and role class.

```
PREFIX cap: <https://hcis.io/ontology/aicapstone/2026/>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT DISTINCT ?obj ?label ?role
WHERE {
  ?obj a cap:GraspableObject .
  FILTER(STRSTARTS(STR(?obj),
    "https://hcis.io/ontology/aicapstone/2026/group10/"))
  OPTIONAL { ?obj cap:hasObjectLabel ?label . }
  OPTIONAL {
    ?obj cap:hasTaskRole ?roleIndividual .
    VALUES ?role {
      cap:TargetObject
      cap:ReferenceObject
      cap:ContainerTarget
      cap:CollectableObject
    }
    ?roleIndividual a ?role .
  }
```

```
}
ORDER BY ?obj
```

The saved output contains six inferred graspable objects:

```
obj           | label            | role
--------------+------------------+----------------------
g10:block01   | toy_block_red    | cap:CollectableObject
g10:block02   | toy_block_yellow | cap:CollectableObject
g10:blueCup01 | blue_cup         | cap:TargetObject
g10:fork01    | fork             | cap:TargetObject
g10:knife01   | knife            | cap:TargetObject
g10:pinkCup01 | pink_cup         | cap:TargetObject
```

`g10:plate01` and `g10:basket01` are not included. They are task-relevant objects, but they do not have the asserted grasping affordance in this baseline model.

7 Reproducibility

The workflow can be run locally from the repository root:

```
uv sync
uv run python src/run_reasoning.py
uv run pytest
```

The tests verify that Turtle files parse, Group 10 objects are not manually asserted as `cap:GraspableObject`, task-role values are Group 10 role individuals, expected objects are inferred as graspable, and saved query output matches the expected result.

8 Limitations

This submission intentionally stays within the baseline task scope. It does not model advanced gripper constraints, mass, deformability, task success criteria, or policy success/failure comparisons. The reasoning workflow is designed as a compact educational implementation rather than a complete OWL DL reasoning system.